PROJECT SPECIFICATION

Assignment 1: AIM TRAINER

This document describes the project specification for students in the Game Programming course. The goal is to become familiar with the game loop, input handling, timer/time management, and HUD design.

Library:	pygame (recommended) or equivalent simple graphics/input library
Submission Deadline:	**Week 02/03/2026**
Submission:	Presentation + Github link containing source code + README

1 Game Overview

Aim Trainer is a mini-game for reflex training: each round, targets appear at a random position. The player must click as quickly and accurately as possible before the target disappears. The game runs for a fixed duration and displays final results.

Topic focus: game loop, spawn/despawn using timers, mouse input, reaction time measurement, HUD, and state machine.

Learning Outcomes

In this assignment, students are required to:

- Build a stable game loop (update/draw) and understand the concept of frame time (dt).
- Handle input (mouse clicks, keyboard shortcuts) and manage states (START / PLAYING / PAUSED / RESULTS).
- Use timers to spawn/despawn targets, calculate reaction time, and control difficulty.
- Design a HUD that clearly displays gameplay statistics.

Technical Requirements and Setup

Using pygame framework is recommended for the simplicity of the first game project. Other game framework/engine is also allowed if the team **has** experience or is willing to challenge themselves.

Students **are advised to use** external assets but have to cite sources in README.

2 Functional Requirements

This section specifies the minimum functional requirements for the Aim Trainer game. Students may extend beyond the requirements, but the MVP must satisfy all items below.

1. **Game Flow and Screens**
   - **Start screen:** Must include the following buttons: *Start Game, Instruction, Settings, Exit Game.* (Statistics button is optional for MVP).
   - **Gameplay:** The game runs for a **Fixed duration of 60 seconds**. Implementing configurable duration is part of the Bonus/Settings.
   - **Results:** When the timer ends, transition to a results screen and show the **summary of the current session** (Score, Hits, Misses, Accuracy).
2. **Target Spawn/Despawn (MVP)**
   - MVP requires **only 1 target active at a time**. Students are encouraged to design additional aim-training formats (bonus).
   - Target spawns at a **random position** (x, y) that satisfies:

     $$r < x < (\text{Width} - r) \quad \text{and} \quad r < y < (\text{Height} - r)$$

     This ensures the target is fully visible and does not clip the edges.
   - Target has a radius r and a **time-to-live (TTL)** in milliseconds.
   - If TTL expires without being hit: count as a **miss (timeout)** and spawn a new target.

- **Optional:** Add a **100–250 ms delay** between rounds to create rhythm.

3. **Input and Hit Test (No Collision Engine Required)**
   - On **left mouse click**, determine whether the click position lies within the target circle.
4. **Reaction Time and Performance Metrics**
   - **Reaction time** is measured from the moment the target appears to the moment it is hit.
   - The game must track (at minimum): **Hits, Misses, Score**.
   - Recommended additional metrics:
     - Accuracy (%).
     - Average reaction time (ms).
     - Best (minimum) reaction time (ms).
5. **Scoring and HUD Display**
   - **Minimum HUD during gameplay:**
     - Time left (seconds or mm:ss).
     - Score.
     - Hits / Misses.
   - **Scoring formula:** Students may implement any reasonable formula, but it must be explained in the README. A simple suggested formula (beginner-friendly):
     - Each hit: + `base_points` (e.g., 100).
     - Reflex bonus:
       $$\text{bonus} = \left\lfloor \frac{\max(0, \text{TTL} - \text{ReactionTime})}{\text{TTL}} \times \text{bonus_cap} \right\rfloor$$
       (e.g., `bonus_cap` = 50).
     - Miss (missed click or timeout): +1 miss (may subtract points slightly or not at all).

```python
# Example Python implementation
score += 100 + int(max(0, ttl - reaction) / ttl * 50)
```

6. **Difficulty Progression**
   - The game should become harder over time to avoid flat gameplay. Possible mechanisms include:
     - Gradually decrease TTL (targets disappear faster).
     - Gradually decrease radius r (targets become smaller).
     - Increase reward for fast hits (e.g., higher `bonus_cap`) while TTL decreases.
   - Students may implement other progression designs beyond these suggestions.
7. **Game States and User Experience**
   - Minimum required states:
     - **START:** show instructions and start button/key.
     - **PLAYING:** gameplay + HUD.
     - **RESULTS:** show session summary + restart/quit options.
     - **PAUSED (ESC):** pause gameplay and display an overlay.
   - **Hit/miss feedback:** provide clear feedback such as target color change, a pop sound, or floating text (e.g., +points).

3 Submission Requirements

- Upload the project to a public github link and submit the link to LMS.
- README must include: how to run, controls, game rules and asset sources.
- Prepare to present the project to the class and answer instructor's questions.

3.1 Bonus Extensions

Below are some extensions that students may implement if want to get bonus score.

- **Combo system:** consecutive hits increase multiplier, miss resets combo.
- **Interesting gameplay mode:** aim training mode that is challenging to implement.
- **Persistent Statistics:** Save results of each round to file/database and show historical stats of player (Graphs, Best records, etc.).
- **Settings:** choose game duration, difficulty, target size, mouse-related settings.

End of document.